```
sudo mkdir /frog
sudo chown reddatabase /frog
sudo -u sysdba gfix create /test/МебельМаг/мебель.fdb -user SYSDBA -password masterkey
sudo chown $USER:$USER /test/МебельМаг/мебель.fdb
-- Таблица пользователей
CREATE TABLE USERS (
    ID INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ROLE VARCHAR(50) NOT NULL,
    FULL_NAME VARCHAR(150) NOT NULL,
    LOGIN VARCHAR(100) NOT NULL UNIQUE,
    PASSWORD VARCHAR(100) NOT NULL
);

-- Таблица товаров (расширенная)
CREATE TABLE PRODUCT (
    PRODUCT_ARTICLE VARCHAR(7) PRIMARY KEY,
    PRODUCT_NAME VARCHAR(100) NOT NULL,
    DESCRIPTION BLOB SUB_TYPE TEXT,
    CATEGORY VARCHAR(100),
    MANUFACTURER VARCHAR(100),
    SUPPLIER VARCHAR(100) NOT NULL,
    PRICE DECIMAL(15,2) NOT NULL,
    UNIT VARCHAR(20) NOT NULL,
    STOCK_QUANTITY INTEGER NOT NULL,
    DISCOUNT DECIMAL(5,2) DEFAULT 0,
    IMAGE_PATH VARCHAR(500)
);

-- Таблица заказов
CREATE TABLE ORDERS (
    ORDER_ID INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    USER_ID INTEGER NOT NULL,
    ORDER_DATE TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    STATUS VARCHAR(50),
    FOREIGN KEY (USER_ID) REFERENCES USERS(ID)
);

-- Таблица позиций заказа
CREATE TABLE ORDER_ITEMS (
    ORDER_ITEM_ID INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ORDER_ID INTEGER NOT NULL,
    PRODUCT_ARTICLE VARCHAR(7) NOT NULL,
    QUANTITY INTEGER NOT NULL,
    FOREIGN KEY (ORDER_ID) REFERENCES ORDERS(ORDER_ID),
    FOREIGN KEY (PRODUCT_ARTICLE) REFERENCES PRODUCT(PRODUCT_ARTICLE)
);
```

```
INSERT INTO USERS (ROLE, FULL_NAME, LOGIN, PASSWORD) VALUES
('Администратор', 'Иванов Иван Иванович', 'admin', 'admin123'),
('Менеджер', 'Петрова Анна Сергеевна', 'manager', 'manager123'),
('Авторизованный клиент', 'Сидоров Алексей', 'client', 'client123');

INSERT INTO PRODUCT (PRODUCT_ARTICLE, PRODUCT_NAME, DESCRIPTION, CATEGORY, MANUFACTURER, SUPPLIER, PRICE,
UNIT, STOCK_QUANTITY, DISCOUNT, IMAGE_PATH) VALUES
('FURN001', 'Стол письменный "Директор"', 'Дуб, металлические ножки', 'Офисная мебель', 'МебельМастер', 'ООО МебельОпт', 12500.00, 'шт'
, 10, 30, NULL),
('FURN002', 'Кресло офисное', 'Кожзам, регулировка высоты', 'Кресла', 'ComfortLine', 'ИП Иванов', 8500.50, 'шт', 0, 10, NULL),
('FURN003', 'Шкаф для документов', 'ЛДСП, 3 полки', 'Хранение', 'МебельМастер', 'ООО МебельОпт', 7200.00, 'шт', 5, 0, NULL);

INSERT INTO ORDERS (USER_ID, STATUS) VALUES (3, 'Новый');
INSERT INTO ORDER_ITEMS (ORDER_ID, PRODUCT_ARTICLE, QUANTITY) VALUES (1, 'FURN001', 2);

COMMIT;
```

4.1 DatabaseService.cs (расширенный)

csharp

```
using Microsoft.EntityFrameworkCore;
using МебельМаг.Entities;

namespace МебельМаг.Services
{
    public class DatabaseService : IDisposable
    {
        private readonly МебельМагContext _context;
        private static readonly object _dbLock = new object();

        public DatabaseService()
        {
            _context = new МебельМагContext();
        }

        // Аутентификация (как в методичке, но можно добавить хеширование)
        public async Task<User?> AuthenticateUserAsync(string login, string password)
        {
            lock (_dbLock)
            {
                return _context.Users
                    .AsNoTracking()
                    .FirstOrDefault(u => u.Login == login && u.Password == password);
            }
        }
```

```
// Получение товаров с поиском, сортировкой и фильтром по скидке
public async Task<ObservableCollection<Product>> GetProductsAsync(
    string filter = "",
    string sortBy = "NAME_ASC",
    string discountRange = "ALL")
{
    IQueryable<Product> query = _context.Products.AsNoTracking();

    // Поиск по текстовым полям
    if (!string.IsNullOrWhiteSpace(filter))
    {
        query = query.Where(p =>
            p.ProductName.Contains(filter) ||
            p.ProductArticle.Contains(filter) ||
            p.Description.Contains(filter) ||
            p.Category.Contains(filter) ||
            p.Manufacturer.Contains(filter) ||
            p.Supplier.Contains(filter));
    }

    // Фильтр по скидке
    if (discountRange != "ALL")
    {
        query = discountRange switch
        {
            "0-12.99" => query.Where(p => p.Discount >= 0 && p.Discount <= 12.99m),
            "13-16.99" => query.Where(p => p.Discount >= 13 && p.Discount <= 16.99m),
            "17+" => query.Where(p => p.Discount >= 17),
            _ => query
        };
    }

    // Сортировка
    query = sortBy switch
    {
        "NAME_DESC" => query.OrderByDescending(p => p.ProductName),
        "PRICE_ASC" => query.OrderBy(p => p.Price),
        "PRICE_DESC" => query.OrderByDescending(p => p.Price),
        "STOCK_ASC" => query.OrderBy(p => p.StockQuantity),
        "STOCK_DESC" => query.OrderByDescending(p => p.StockQuantity),
        _ => query.OrderBy(p => p.ProductName)
    };

    var products = query.ToList();
    return new ObservableCollection<Product>(products);
}
```

```
    // Удаление с проверкой наличия в заказах
    public async Task<(bool success, string message)> DeleteProductWithCheckAsync(string productArticle)
    {
        lock (_dbLock)
        {
            var inOrder = _context.OrderItems.Any(oi => oi.ProductArticle == productArticle);
            if (inOrder)
                return (false, "Товар присутствует в заказах и не может быть удалён.");

            var product = _context.Products.Find(productArticle);
            if (product != null)
            {
                if (!string.IsNullOrEmpty(product.ImagePath) && File.Exists(product.ImagePath))
                    File.Delete(product.ImagePath);

                _context.Products.Remove(product);
                _context.SaveChanges();
                return (true, "Товар удалён.");
            }
            return (false, "Товар не найден.");
        }
    }

    // Добавление и обновление товара (как в методичке, но с ImagePath)
    public async Task<bool> AddProductAsync(Product product)
    {
        lock (_dbLock)
        {
            _context.Products.Add(product);
            _context.SaveChanges();
            return true;
        }
    }

    public async Task<bool> UpdateProductAsync(Product product)
    {
        lock (_dbLock)
        {
            _context.Products.Update(product);
            _context.SaveChanges();
            return true;
        }
    }

    public void Dispose() => _context?.Dispose();
  }
}
```

4.2 ImageService.cs (работа с фото)

csharp

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

namespace МебельМаг.Services
{
    public class ImageService
    {
        public string SaveImage(string sourcePath, string productArticle)
        {
            string targetDir = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Images");
            if (!Directory.Exists(targetDir))
                Directory.CreateDirectory(targetDir);

            using var image = Image.Load(sourcePath);
            image.Mutate(x => x.Resize(new ResizeOptions
            {
                Size = new Size(300, 200),
                Mode = ResizeMode.Max
            }));

            string ext = Path.GetExtension(sourcePath);
            string targetPath = Path.Combine(targetDir, $"{productArticle}{ext}");
            image.Save(targetPath);
            return targetPath;
        }

        public byte[] LoadImageBytes(string path)
        {
            if (File.Exists(path))
                return File.ReadAllBytes(path);
            return null;
        }
    }
}
```

Этап 5. ViewModels

5.1 MainWindowViewModel.cs (добавлен гость)

csharp

```
public class MainWindowViewModel : ViewModelBase
```

```
{
    private ViewModelBase _contentViewModel;

    public MainWindowViewModel()
    {
        _contentViewModel = new LoginViewModel(this);
    }

    public ViewModelBase ContentViewModel
    {
        get => _contentViewModel;
        private set => this.RaiseAndSetIfChanged(ref _contentViewModel, value);
    }

    public void LoginSuccessful(User user)
    {
        ContentViewModel = new ProductListViewModel(user, this);
    }

    public void GuestLogin()
    {
        var guestUser = new User { Role = "Гость", FullName = "Гость" };
        ContentViewModel = new ProductListViewModel(guestUser, this);
    }

    public void Logout()
    {
        ContentViewModel = new LoginViewModel(this);
    }

    public void NavigateToProductEdit(Product product = null)
    {
        ContentViewModel = new ProductEditViewModel(product, this);
    }
}
```

5.2 ProductListViewModel.cs (ключевой — роли, подсветка, команды)

csharp

```
public class ProductListViewModel : ViewModelBase
{
    private readonly User _currentUser;
    private readonly MainWindowViewModel _mainVM;
    private readonly DatabaseService _dbService;

    private ObservableCollection<Product> _products;
    private string _searchFilter = "";
```

```
    private string _selectedSort = "NAME_ASC";
    private string _selectedDiscountRange = "ALL";
    private bool _isLoading;

    public ProductListViewModel(User user, MainWindowViewModel mainVM)
    {
        _currentUser = user;
        _mainVM = mainVM;
        _dbService = new DatabaseService();

        _ = LoadProductsAsync();

        // Подписки на изменения
        this.WhenAnyValue(x => x.SearchFilter)
            .Throttle(TimeSpan.FromMilliseconds(500))
            .Subscribe(async _ => await UpdateProductsAsync());
        this.WhenAnyValue(x => x.SelectedSort)
            .Subscribe(async _ => await UpdateProductsAsync());
        this.WhenAnyValue(x => x.SelectedDiscountRange)
            .Subscribe(async _ => await UpdateProductsAsync());

        // Команды
        LogoutCommand = ReactiveCommand.Create(() => _mainVM.Logout());
        AddProductCommand = ReactiveCommand.Create(() => _mainVM.NavigateToProductEdit(null));
        EditProductCommand = ReactiveCommand.Create<Product>(p => _mainVM.NavigateToProductEdit(p));
        DeleteProductCommand = ReactiveCommand.CreateFromTask<Product>(ExecuteDeleteProductAsync);
    }

    // Свойства
    public ObservableCollection<Product> Products { get => _products; set => this.RaiseAndSetIfChanged(ref _products, value); }
    public string SearchFilter { get => _searchFilter; set => this.RaiseAndSetIfChanged(ref _searchFilter, value); }
    public string SelectedSort { get => _selectedSort; set => this.RaiseAndSetIfChanged(ref _selectedSort, value); }
    public string SelectedDiscountRange { get => _selectedDiscountRange; set => this.RaiseAndSetIfChanged(ref _selectedDiscountRange, value); }
    public bool IsLoading { get => _isLoading; set => this.RaiseAndSetIfChanged(ref _isLoading, value); }

    public bool IsAdmin => _currentUser.Role == "Администратор";
    public bool IsManager => _currentUser.Role == "Менеджер";
    public bool CanFilter => IsAdmin || IsManager;
    public string WelcomeText => $"Добро пожаловать, {_currentUser.FullName} ({_currentUser.Role})";

    public List<string> SortDisplayNames => new() { "По имени (А-Я)", "По имени (Я-А)", "По цене (↑)", "По цене (↓)", "По остатку (↑)", "По ос
татку (↓)" };
    public List<string> DiscountDisplayNames => new() { "Все скидки", "0-12.99%", "13-16.99%", "17% и более" };

    public ReactiveCommand<Unit, Unit> LogoutCommand { get; }
    public ReactiveCommand<Unit, Unit> AddProductCommand { get; }
    public ReactiveCommand<Product, Unit> EditProductCommand { get; }
```

```
public ReactiveCommand<Product, Unit> DeleteProductCommand { get; }

// Подсветка строки (вызывается из View через binding)
public string GetRowBackground(Product product)
{
    if (product.StockQuantity == 0) return "#D3D3D3"; // серый
    if (product.Discount > 25) return "#23E1EF";
    return "#FFFFFF";
}

public string GetPriceDisplay(Product product)
{
    if (product.Discount > 0)
    {
        decimal finalPrice = product.Price * (1 - product.Discount / 100);
        return $"~~{product.Price:F2}~~ руб. → {finalPrice:F2} руб.";
    }
    return $"{product.Price:F2} руб.";
}

private async Task LoadProductsAsync()
{
    IsLoading = true;
    Products = await _dbService.GetProductsAsync("", "NAME_ASC", "ALL");
    IsLoading = false;
}

private async Task UpdateProductsAsync()
{
    if (CanFilter)
        Products = await _dbService.GetProductsAsync(SearchFilter, SelectedSort, SelectedDiscountRange);
    else
        Products = await _dbService.GetProductsAsync("", "NAME_ASC", "ALL");
}

private async Task ExecuteDeleteProductAsync(Product product)
{
    var result = await _dbService.DeleteProductWithCheckAsync(product.ProductArticle);
    await ShowMessage(result.success ? "Успех" : "Ошибка", result.message);
    if (result.success)
        await UpdateProductsAsync();
}

private async Task ShowMessage(string title, string msg)
{
    var box = MessageBox.Avalonia.MessageBoxManager.GetMessageBoxStandardWindow(title, msg);
    await box.ShowDialog((Window)Application.Current.ApplicationLifetime.MainView);
```

```
  }
}
```